

Relazione tecnica

Academic Paper Analyzer

*Documentazione tecnica di supporto alla tesi:
architettura, scelte progettuali e pipeline di analisi*

Riccardo Aldrovandi

Repository: paper_analyzer

Settembre 2026

Indice

1. Visione d'insieme	2
2. <code>`main.py`</code> — l'orchestratore	2
2.1 Funzioni di supporto	3
2.2 I sei sottocomandi	3
3. <code>`src/config.py`</code> — configurazione e system prompt	4
4. <code>`src/api/`</code> — ricerca dei metadati	4
5. <code>`src/download/downloader.py`</code> — download effettivo	5
6. <code>`src/processing/`</code> — estrazione di testo e immagini	5
7. <code>`src/advanced/`</code> — la pipeline visuo-matematica	6
8. <code>`src/llm/`</code> — i tre provider LLM	7
9. <code>`src/utils/`</code> — utilità trasversali	8
10. Flusso end-to-end: <code>`full-analyze`</code> ed <code>`export-dataset`</code>	8
11. Robustezza della pipeline: un riepilogo trasversale	9
12. Dipendenze principali (<code>`requirements.txt`</code>)	9

Questa relazione descrive in dettaglio come è fatto e come funziona il codice del progetto, file per file. È pensata come documentazione di supporto alla tesi: spiega cosa fa ogni componente, perché è stato progettato così, e come i pezzi si incastrano tra loro nella pipeline complessiva.

1. Visione d'insieme

Il progetto è un framework a riga di comando (*main.py*) che automatizza l'intero ciclo di una revisione della letteratura scientifica:

1. **Cerca** paper su Semantic Scholar e INSPIRE HEP, e ne scarica PDF e sorgenti LaTeX da arXiv.
2. **Estrae** il testo di un paper (da PDF o da sorgente LaTeX) e le sue figure/grafici.
3. **Digitalizza** ogni grafico con un modello di visione (Gemini Vision, con fallback su OpenRouter), separando le curve sovrapposte in serie distinte.
4. **Scopre la formula matematica** dietro ogni curva continua tramite regressione simbolica (PySR), validando ogni equazione su un sottoinsieme di punti mai visto durante la ricerca, per scartare formule che inseguono il rumore invece di descrivere il vero andamento.
5. **Arricchisce** il testo del paper con queste scoperte visive, mantenendole esplicitamente separate dai risultati dichiarati dagli autori.
6. **Interroga un LLM** (Gemini, Groq o OpenRouter) per produrre un riassunto strutturato in JSON del paper (contributi, metodologia, risultati, limiti, ecc.).
7. **Esporta** tutto questo come dataset in formato chat-SFT, pensato per il fine-tuning di un modello locale (Qwen/Llama) che un giorno sostituirà le chiamate API a pagamento.

Il codice è organizzato per separazione di responsabilità: ogni cartella sotto *src/* si occupa di un solo aspetto della pipeline, e *main.py* fa da orchestratore che le mette in sequenza tramite i sei sottocomandi della CLI.

```
paper_analyzer/  
├─ main.py                # entry point CLI, orchestrazione dei 6 sottocomandi  
├─ src/  
│   ├─ api/               # ricerca metadati su Semantic Scholar e INSPIRE HEP  
│   ├─ download/          # download effettivo di PDF e sorgenti LaTeX  
│   ├─ processing/        # estrazione testo e immagini da PDF/LaTeX  
│   ├─ advanced/          # digitalizzazione grafici + regressione simbolica  
│   ├─ llm/               # client per i tre provider LLM (Gemini, Groq, OpenRouter)  
│   ├─ utils/             # helper generici (sanitizzazione, export, retry)  
│   └─ config.py          # chiavi API, percorsi, prompt di sistema per l'LLM  
└─ requirements.txt
```

2. `main.py` — l'orchestratore

È il punto di ingresso del programma e contiene sia la definizione della CLI (via *argparse*) sia tre

funzioni di supporto condivise da più comandi, per evitare di duplicare la stessa logica in punti diversi.

2.1 Funzioni di supporto

run_llm_providers(text_content, model_arg) Centralizza il ciclo "manda il testo a uno o più LLM e stampa il JSON risultante". Se *model_arg* == "all" gira su Gemini, Groq e OpenRouter in sequenza, altrimenti solo sul provider richiesto. È condivisa tra *analyze* e *full-analyze*, così i due comandi restituiscono i risultati esattamente nello stesso formato.

analyze_figure(image_path, iterations) È il cuore della pipeline visuo-matematica applicata a una singola immagine. Chiama *digitize_plot()* per trasformare il grafico in dati strutturati; se il grafico è continuo, per ogni curva rilevata chiama *discover_formula()* (PySR) e costruisce una riga di testo con l'equazione trovata e le metriche di affidabilità (R^2 su fit e su holdout, con un avviso esplicito se il gap tra i due è troppo grande — segno di overfitting). Se il grafico è discreto, riporta invece il riassunto testuale e la mappa punto → etichetta prodotta dal digitalizzatore. Qualunque errore (immagine illeggibile, curva che non converge, ecc.) viene loggato e la figura viene semplicemente saltata: una figura rotta non deve far fallire l'analisi dell'intero paper.

build_enriched_paper_text(pdf, latex, iterations, max_figures) Mette insieme tutto il lato "visivo" della pipeline: estrae il testo del paper, estrae le figure, le analizza una per una con *analyze_figure()*, e infine appende in coda al testo una sezione --- *ADVANCED PLOT ANALYSIS* --- con un disclaimer esplicito che ricorda come quelle formule **non** facciano parte del paper originale ma siano scoperte indipendenti dell'algoritmo. Il parametro *max_figures* mette un tetto al numero di figure effettivamente analizzate per paper: senza questo limite, un paper con decine di grafici ripetitivi (tipico delle appendici con ablation study) può da solo esaurire la quota API giornaliera. Questa funzione è usata sia da *full-analyze* sia da *export-dataset*, così i due comandi non possono mai comportarsi in modo diverso sullo stesso input.

already_exported_sources(output_path) Legge un dataset JSONL già esistente e restituisce l'insieme dei file sorgente già processati, leggendo semplicemente il campo *source_file* di ogni riga. È il meccanismo che rende *export-dataset* **riprendibile**: se lo interrompi o lo rilanci, non risponde chiamate API sui paper già fatti.

2.2 I sei sottocomandi

Comando	Cosa fa	Tocca la pipeline visiva?
<code>`fetch`</code>	Cerca paper su Semantic Scholar/INSPIRE, scarica PDF (sempre) e LaTeX (se <code>`--latex`</code>)	No
<code>`analyze`</code>	Estrae solo il testo di un singolo paper e lo manda all'LLM	No
<code>`images`</code>	Estrae solo le figure da un singolo paper	Solo estrazione, non analisi
<code>`advanced`</code>	Digitalizza e trova la formula di un singolo grafico già estratto	Sì (di per sé)
<code>`full-analyze`</code>	Pipeline completa su un singolo paper: testo + figure + formule + LLM	Sì

Comando	Cosa fa	Tocca la pipeline visiva?
<code>`export-dataset`</code>	Come <code>`full-analyze`</code> , ma in batch su tutti i paper scaricati, salvando gli input/output come dataset JSONL	Sì

fetch interroga `fetch_semantic_scholar()` e/o `fetch_inspire_hep()` a seconda di `api` (`semantic/inspire/both`), applica un filtro opzionale sul numero minimo di citazioni (`min-citations`) e, solo per INSPIRE, un ordinamento (`sort_mostrecent|mostcited`) che viene passato direttamente come parametro `sort` all'API REST di INSPIRE. I risultati vengono salvati in JSON e CSV tramite `export_results()`.

analyze è la versione "leggera": estrae il testo con `extract_text_from_pdf/extract_text_from_latex_tarball` e lo manda diretto a `run_llm_providers()`, senza mai toccare immagini o PySR.

images espone da riga di comando l'estrazione di figure (`extract_images_from_pdf/extract_images_from_latex`), utile per ispezionare a occhio cosa il programma riesce a isolare da un paper prima di lanciare l'analisi vera e propria.

advanced è il banco di prova della pipeline visuo-matematica su una singola immagine già estratta: chiama `digitize_plot()` e poi, per ogni curva continua trovata, `discover_formula()`, stampando equazione, complessità e le due metriche R^2 con l'eventuale avviso di overfitting.

full-analyze e **export-dataset** sono descritti nel dettaglio nella sezione 6 (flusso end-to-end), dato che sono quelli che mettono in gioco tutti i moduli contemporaneamente.

3. ``src/config.py`` — configurazione e system prompt

File piccolo ma centrale. Carica le quattro chiavi API da `.env` (`S2_API_KEY`, `GEMINI_API_KEY`, `GROQ_API_KEY`, `OPENROUTER_API_KEY`) tramite `python-dotenv`, definisce le directory di destinazione dei download (`downloaded_pdfs/`, `downloaded_latex/`, create automaticamente se mancanti), e soprattutto contiene `SYSTEM_PROMPT`: il prompt che viene anteposto al testo di ogni paper in ogni chiamata LLM.

Il prompt definisce lo schema JSON che l'LLM deve restituire (10 campi: problema affrontato, contributi, metodologia, dataset usati, esperimenti, risultati, limiti, sviluppi futuri, elementi rilevanti, e `visual_analysis_findings`) e contiene un'istruzione esplicita e piuttosto insistita sul fatto che tutto ciò che si trova nella sezione `--- ADVANCED PLOT ANALYSIS ---` del testo (le formule scoperte da PySR) **non va mai confuso con i risultati del paper**: deve finire solo nel campo dedicato `visual_analysis_findings`, mai in `results` o `methodology`. Questa regola è stata aggiunta dopo aver osservato empiricamente che, senza un campo separato nello schema, l'LLM tendeva ad attribuire agli autori del paper formule che in realtà erano approssimazioni dell'algoritmo (vedi sezione 8).

4. ``src/api/`` — ricerca dei metadati

`semantic_scholar.py` — **`fetch_semantic_scholar(query, limit, fetch_latex, min_citations)`** Interroga l'endpoint pubblico `api.semanticscholar.org/graph/v1/paper/search`. Ha una gestione robusta del rate limiting: se riceve un 429 riprova fino a 5 volte con backoff esponenziale ($3^{\text{tentativo}}$ secondi). Per ogni risultato estrae titolo, autori, anno, citazioni, ArXiv ID (dentro `externalIds`) e link PDF ad accesso aperto; se `min_citations` è impostato, scarta subito i paper sotto soglia prima ancora di tentare il download (evita download inutili). Il download vero e proprio è delegato a `download_pdf()/download_latex()` di `src/download/downloader.py`.

`inspire_hep.py` — **`fetch_inspire_hep(query, limit, fetch_latex, min_citations, sort_by)`** Stessa logica applicata all'API di INSPIRE HEP (`inspirehep.net/api/literature`), specializzata in fisica delle alte energie. In più supporta il parametro `sort_by` (`mostrecent/mostcited`), inoltrato direttamente come parametro `sort` della query REST — utile perché su INSPIRE una ricerca generica per argomento tende a restituire prima i paper storici/fondativi più citati in assoluto piuttosto che quelli recenti, e viceversa senza uno `min-citations` alto si rischia di pescare risultati poco rilevanti. L'ArXiv ID qui si trova in `metadata.arxiv_eprints`, e **non è sempre presente**: molti paper indicizzati da INSPIRE sono più vecchi di arXiv stesso (nato nel 1991) e quindi non hanno alcun preprint da scaricare — è un limite dei dati, non un bug (verificato empiricamente, vedi sezione 8).

5. ``src/download/downloader.py`` — download effettivo

Due funzioni simmetriche, entrambe con lo stesso pattern difensivo: se il file esiste già in locale, saltano il download (`export-dataset` e i `fetch` ripetuti sfruttano proprio questo per essere idempotenti).

`download_pdf(pdf_url, paper_id, title)` scarica il PDF con un User-Agent da browser (alcuni server accademici bloccano richieste senza) e `verify=False` per bypassare certificati scaduti tipici di alcuni server universitari. Salva solo se il `Content-Type` della risposta è effettivamente `application/pdf`.

`download_latex(arxiv_id, paper_id, title)` scarica il sorgente da `arxiv.org/e-print/<id>`. Qui c'è un controllo aggiunto in un secondo momento: arXiv a volte risponde con un PDF invece che con un vero archivio gzip, quando l'autore non ha caricato il sorgente LaTeX separato. La funzione verifica sia il `Content-Type` sia i primi due byte del contenuto (`\x1f\x8b`, la firma magica di gzip) prima di salvare il file come `.tar.gz`: se non è davvero un archivio gzip, ritorna "N/A" onestamente invece di salvare un file dal nome ingannevole che poi fallirebbe silenziosamente in fase di estrazione (vedi sezione 8, era un bug reale già osservato su un caso concreto).

6. ``src/processing/`` — estrazione di testo e immagini

`pdf_extractor.py` — **`extract_text_from_pdf(pdf_path)`** Il più semplice dei tre: apre il PDF con `PyMuPDF`

e concatena il testo pagina per pagina (`page.get_text("text")`). Nessuna gestione di layout complessi o OCR — se il PDF è una scansione senza testo selezionabile, il risultato sarà vuoto o quasi.

`latex_extractor.py` — **`extract_text_from_latex_tarball(tar_path)`** Apre l'archivio con il modulo `tarfile` in modalità `r:gz`, e per ogni file che termina in `.tex` ne legge il contenuto (provando prima UTF-8, poi latin-1 come fallback) concatenandolo con un'intestazione che indica il nome del file sorgente. Tutta l'apertura dell'archivio è avvolta in un `try/except`: se il file non è un vero archivio gzip (vedi il bug di cui sopra), la funzione non solleva eccezioni verso l'alto ma stampa un errore e ritorna una stringa vuota — comportamento silenzioso ma sicuro, assorbito più a valle dal controllo "salta se il testo è vuoto" presente in `export-dataset`.

`image_extractor.py` — il modulo più elaborato dei tre, perché non si limita a estrarre ma applica un filtro intelligente.

- **`extract_images_from_pdf(pdf_path)`**: per ogni pagina del PDF, estrae prima le immagini raster incorporate direttamente (scartando quelle sotto i 200×200 pixel, tipicamente loghi/icone). Poi, se la pagina menziona testualmente "fig"/"figure"/"plot", ne renderizza un'immagine a piena pagina e la passa a `_is_actual_plot()`: una chiamata a Gemini Vision che risponde SÌ/NO alla domanda "questa pagina contiene davvero un grafico con dati numerici, o è solo testo/tabelle/diagrammi architetturali?". Solo se la risposta è affermativa la pagina passa a `_crop_individual_plots()`, che chiede a Gemini Vision le bounding box di ogni grafico distinto presente sulla pagina (in coordinate normalizzate 0-1000) e le ritaglia singolarmente ad alta risoluzione (300 DPI), rimuovendo il render dell'intera pagina una volta che il ritaglio è riuscito.
- **`extract_images_from_latex(tar_path)`**: più diretta, perché i sorgenti LaTeX contengono già le figure come file separati (`.png`, `.jpg`, `.pdf`, `.eps`); i formati vettoriali vengono convertiti in PNG a 300 DPI. Non applica il filtro Gemini, perché ogni immagine estratta è per definizione una figura del paper (non una pagina intera da giudicare).

Un dettaglio di design importante: nella funzione `_is_actual_plot()`, se la quota Gemini è esaurita, il codice **non** delega la decisione al modello di fallback più debole (OpenRouter) — semplicemente salta il filtro e tiene la pagina. Il ragionamento: qui un falso "NO" scarta la figura per sempre e silenziosamente, mentre un falso "SÌ" costa solo un tentativo in più a valle (che ha comunque un suo fallback sicuro). È una scelta di sicurezza asimmetrica, aggiunta dopo aver osservato in pratica che il modello gratuito di fallback etichettava erroneamente come "non un grafico" un'immagine che invece lo era (vedi sezione 8).

7. ``src/advanced/`` — la pipeline visuo-matematica

`digitizer.py` — **`digitize_plot(image_path)`** Manda l'immagine a Gemini Vision con un prompt che chiede di: (1) classificare il grafico come continuo o discreto; (2) se continuo, identificare **ogni curva distinta** (per colore/legenda) ed estrarre 15-30 coppie (X, Y) per ciascuna, senza mai mescolare punti di curve diverse; (3) se discreto, estrarre i singoli punti con le rispettive etichette testuali. Questo è un punto critico del progetto: la prima versione chiedeva di estrarre punti dalla "curva principale" al singolare, e

quando un grafico aveva più linee sovrapposte (es. il confronto tra tre varianti di un modello) il risultato era un insieme di coordinate "Frankenstein" mescolate tra curve diverse, che poi PySR interpolava producendo equazioni prive di senso.

I punti di ogni curva vengono ripuliti da `_clean_points()`: ordinati per X e deduplicati (tenendo la prima occorrenza), perché coordinate fuori ordine o duplicate confondono la regressione simbolica più di quanto aiuti avere qualche punto in più.

Se la quota Gemini è esaurita, la funzione passa a `analyze_image_with_openrouter()` (definita in `src/llm/openrouter_client.py`) come fallback, riusando lo stesso prompt.

pysr.py — `discover_formula(X, y, iterations)` Applica la regressione simbolica vera e propria tramite la libreria PySR (motore Julia). Alcune scelte di configurazione non ovvie, tutte emerse da problemi osservati durante i test:

- **Operatori limitati** a `+` `-` `*` `/` e `exp/log` (niente `sin/cos`): con soli 15-30 punti rumorosi (letture approssimative di un grafico), funzioni trigonometriche ad alta frequenza trovano sempre un modo per "passare vicino" al rumore producendo un fit perfetto ma privo di significato.
- **`nested_constraints`** vieta di annidare `exp/log` dentro se stessi (niente `exp(exp(exp(...)))`): anche senza trigonometria, PySR può impilare esponenziali per creare una "gobba" che insegue il rumore invece di descrivere un andamento reale.
- **Validazione via holdout** (`_select_by_holdout`): con almeno 8 punti disponibili, una parte (~25%, minimo 3, sempre lasciando almeno 5 per il fit) viene tenuta fuori dalla ricerca. Ogni equazione sul fronte di Pareto di PySR (non solo quella "migliore" secondo il punteggio interno) viene poi valutata su questi punti mai visti; tra le equazioni con punteggio di holdout vicino al massimo si sceglie la più semplice (rasoio di Occam). È l'equivalente automatico di un umano che confronta a occhio la curva trovata con il grafico originale, e in pratica ha permesso di scartare automaticamente formule che sui dati di training avevano R^2 vicino a 1 ma che, guardate con senso critico, erano chiaramente overfit (es. una "gobba" gaussiana dove il grafico mostrava un andamento monotono).
- **`_r2_score()`** gestisce esplicitamente il caso di target quasi costante (es. una curva di riferimento piatta), dove la formula standard di R^2 è una divisione $0/0$: senza questo caso speciale, un fit perfetto di una costante veniva erroneamente segnalato come `nan/-inf`.
- **`temp_equation_file=True`** evita che PySR lasci su disco una cartella `outputs/<timestamp>/` per ogni singola curva analizzata (con `export-dataset` che ne lancia decine per run, il disco si riempiva rapidamente di file mai riletti).

8. `src/llm/` — i tre provider LLM

Tutti e tre espongono la stessa firma `analyze_with_<provider>(text_content) -> tuple[str, str]` (nome del modello usato, stringa JSON grezza), così `run_llm_providers()` in `main.py` può trattarli in modo intercambiabile.

gemini_client.py usa il modello *gemini-3.6-flash* tramite l'SDK *google-genai*. È l'unico che implementa la resilienza completa: la chiamata passa per *call_gemini_with_retry()* (vedi *src/utils/retry.py*), e se la quota gratuita giornaliera è esaurita (HTTP 429) passa automaticamente a *analyze_with_openrouter()* come fallback, così un'analisi in corso non si interrompe bruscamente solo perché Gemini ha raggiunto il limite.

groq_client.py usa il modello *groq/compound*. Ha due vincoli operativi specifici del provider: tronca il testo in input a 20.000 caratteri (limite pratico del piano gratuito) e attende 15 secondi prima di ogni chiamata per restare dentro il budget di token al minuto.

openrouter_client.py è il più articolato: espone sia *analyze_with_openrouter()* (testo) sia *analyze_image_with_openrouter()* (immagine + prompt, usata come fallback vision da *digitizer.py* e da *image_extractor.py*). Entrambe usano lo stesso alias di modello gratuito *openrouter/free* (centralizzato nella costante *FREE_MODEL*, così l'intero progetto dipende da un'unica definizione di "modello gratuito"). Entrambe le richieste HTTP hanno un timeout di 90 secondi: senza timeout, una risposta lenta del tier gratuito (che capita, essendo l'ultima priorità di OpenRouter) bloccava lo script a tempo indeterminato invece di fallire in modo pulito e passare al paper successivo.

9. `src/utils/` — utilità trasversali

helpers.py: *sanitize_filename()* (rimuove caratteri non validi per nomi file), *export_results()* (salva una lista di dizionari in JSON e CSV), *process_and_print_result()* (stampa il JSON restituito da un LLM, gestendo il caso in cui non sia JSON valido), *strip_json_fences()* (rimuove i blocchi ``json`` che i modelli gratuiti di fallback a volte aggiungono nonostante l'istruzione esplicita di non farlo).

retry.py: contiene *call_gemini_with_retry()* e l'eccezione *QuotaExceededError*. La logica distingue due tipi di errore molto diversi: un 503 (server temporaneamente sovraccarico) viene ritentato fino a 5 volte con backoff lineare (5s, 10s, 15s...), perché è un problema transitorio; un 429 (quota giornaliera esaurita) **non** viene mai ritentato — sollevare *QuotaExceededError* immediatamente e passare al fallback è più efficiente che aspettare inutilmente, dato che la quota non si libera nel giro di pochi secondi.

10. Flusso end-to-end: `full-analyze` ed `export-dataset`

Questi due comandi mettono in fila tutto quanto descritto sopra:

```
build_enriched_paper_text(pdf/latex)
├─ extract_text_from_pdf / extract_text_from_latex_tarball   → testo grezzo
├─ extract_images_from_pdf / extract_images_from_latex        → lista immagini
│   (con filtro/crop Gemini Vision, solo per i PDF)
├─ per ogni immagine (fino a --max-figures):
│   analyze_figure()
│   │   ├─ digitize_plot()      → punti (X,Y) per curva, o punti discreti
│   │   └─ discover_formula()   → equazione + R² fit/holdout, per ogni curva continua
```

```

└─ testo arricchito = testo originale + sezione "ADVANCED PLOT ANALYSIS" (con disclaimer)

run_llm_providers(testo arricchito) / chiamata diretta ai provider (export-dataset)
└─ analyze_with_gemini/groq/openrouter(testo arricchito)
    → JSON strutturato a 10 campi, con visual_analysis_findings separato dai risultati del paper

```

La differenza tra i due comandi è solo l'ampiezza: *full-analyze* lo fa su un singolo file passato da riga di comando e stampa il risultato a schermo; *export-dataset* lo fa in batch su tutti i file in *downloaded_pdfs/downloaded_latex/*, salvando ogni coppia (testo arricchito, risposta LLM) come riga JSONL in formato chat-SFT (`{"messages": [{"role": "system", ...}, {"role": "user", ...}, {"role": "assistant", ...}]}`), pensato per essere usato direttamente come dataset di fine-tuning. *export-dataset* è inoltre riprendibile (salta i paper già presenti nell'output) e scarta automaticamente le risposte vuote o non-JSON, così un fallimento su un singolo paper non produce righe spazzatura nel dataset.

11. Robustezza della pipeline: un riepilogo trasversale

Diversi meccanismi di resilienza attraversano più moduli e meritano di essere visti insieme:

- **Retry su errori transitori** (503): implementato una sola volta in *src/utils/retry.py* e riusato da tutte e tre le chiamate Gemini Vision (*image_extractor.py*, *digitizer.py*) più la chiamata di analisi testuale (*gemini_client.py*).
- **Fallback su quota esaurita** (429): passa automaticamente da Gemini a OpenRouter free, sempre tramite lo stesso alias *FREE_MODEL* centralizzato, in quattro punti distinti della pipeline (filtro pagina, ritaglio figure, digitalizzazione grafico, analisi testuale finale) — con l'unica eccezione intenzionale del filtro "è un grafico sì/no", dove il fallback è stato disattivato per il motivo di sicurezza spiegato nella sezione 6.
- **Timeout sulle richieste HTTP dirette** (OpenRouter): evita blocchi a tempo indeterminato quando il tier gratuito è lento a rispondere.
- **Skip granulare invece di interruzione totale**: un'immagine che non si riesce a digitalizzare, una curva su cui PySR fallisce, un paper il cui testo risulta vuoto — vengono tutti loggati e saltati singolarmente, senza mai far fallire l'intero comando *full-analyze/export-dataset* in corso.
- **Validazione statistica invece che visiva**: la scelta dell'equazione finale in *pysr.py* non si basa sulla fiducia nel punteggio interno di PySR, ma su una vera validazione holdout — l'automazione del controllo che altrimenti richiederebbe un umano che guarda ogni grafico.

12. Dipendenze principali (`requirements.txt`)

requests/python-dotenv per HTTP e configurazione; *pymupdf* per il parsing dei PDF; *google-genai* e *groq* come SDK ufficiali dei due provider LLM (OpenRouter è chiamato via *requests* puro, non ha SDK dedicato); *pysr*, *numpy*, *pandas*, *sympy* per la regressione simbolica.

Da notare che l'ambiente di fine-tuning (Unsloth, PyTorch, PEFT, ecc.) **non è in questo file**: vive deliberatamente in un repository/ambiente Python separato (*~/Scrivania/tesi/llm-finetune*), per non mescolare le dipendenze pesanti del training con quelle leggere di questo CLI.